STGNN-Based Microservice Autoscaling and Placement for Fast QoS Recovery at the Edge

Yue Wu, Haitao Zhang, Yepeng Zhang
Beijing Key Lab of Intelligent Telecommunication Software and Multimedia
Beijing University of Posts and Telecommunications, Beijing, China
wuyue@bupt.edu.cn, zht@bupt.edu.cn, zhangyepeng@bupt.edu.cn

***Abstract*—Edge computing enables low-latency data processing by placing resources closer to end users, and integrating microservices enhances scalability and responsiveness. However, maintaining QoS in dynamic, resource-constrained edge environments remains challenging. Existing methods often rely on local load monitoring, overlooking cross-layer load propagation, which leads to delayed autoscaling and prolonged QoS violations. To address this, we propose a Load Propagation-aware Autoscaling and Placement (LPAP) approach for proactive QoS management. LPAP anticipates performance degradation by dynamically adjusting scaling and placement based on both direct and indirect caller impacts and varying resource consumption patterns. It employs a Spatio-Temporal Graph Neural Network (STGNN) to estimate influence coefficients that quantify the impact of invocation edges on microservice performance. These coefficients guide an improved Actor-Critic algorithm that incorporates them into the Temporal Difference (TD) error and reward function to accelerate learning. Experiments show that LPAP effectively reduces QoS degradation duration and improves response times under dynamic workloads.**

***Index Terms*—Edge computing, autoscaling, microservice placement, Quality of Service (QoS), graph neural network**

I. Introduction

Edge computing enhances application performance by bringing computation closer to end users, enabling timely data processing and reduced latency [1]. Microservices architecture [2] decomposes applications into loosely coupled units, offering greater flexibility, scalability, and fault tolerance compared to monolithic systems. Integrating microservices with edge computing improves responsiveness and user experience [3]. However, ensuring QoS in dynamic, resource-constrained edge environments remains challenging. Autoscaling adjusts the number of service replicas based on workload changes and combined with intelligent placement optimizes resource usage and reduces response time [4].

Existing methods face three key challenges in handling cross-layer load propagation. First, most traditional methods [5], [6] make autoscaling decisions based solely on individual microservices, without considering whether upstream load changes affect the QoS of downstream services. This results in delayed scaling and thus prolonged QoS violations. Second, traditional scaling approaches often assume that the same QPS leads to the same resource demand. Existing QoS recovery methods [7], [8] ignore the difference between request sources and simply estimate the number of requests that upstream services will pass to downstream. However, requests from different upstream services can lead to significant variations in a microservice's resource consumption. Third, traditional methods [9] ignore that the request of the same service chain has different resource requirements at different times due to the difference in the amount of data carried. However, a surge in low-data requests may not require additional resources, while high-data requests can significantly increase CPU and memory consumption even if QPS remains constant.

Furthermore, effective placement strategies are essential for ensuring QoS at the edge [4]. Even with accurate autoscaling, network fluctuations can increase communication time between nodes, negatively impacting application response time. Without appropriate placement, QoS violations may persist despite scaling adjustments.

To tackle the challenges, we propose a Load Propagation-aware Autoscaling and Placement (LPAP) approach that jointly optimizes replica scaling and placement. LPAP accounts for cross-layer load propagation and the diverse resource consumption patterns of multiple upstream callers. We first design a Spatio-Temporal Graph Neural Network (STGNN) to estimate influence coefficients, capturing how upstream microservices affect a target microservice using spatial-temporal metrics such as CPU, memory, and request size. By incorporating Graph Attention Networks (GAT), the STGNN learns the significance of upstream interactions and outputs an influence coefficient matrix to guide resource adjustments. Based on this, we develop an improved Actor-Critic algorithm that integrates influence coefficients with dynamic load, resource availability, and network conditions to make adaptive scaling and placement decisions. The policy evolves through learning, enabling timely and effective responses to workload changes. Experimental results show that LPAP achieves faster QoS recovery and reduced response times compared to existing approaches at the edge.

The rest of this paper is organized as follows. Section II introduces system model and problem formulation. In Section III, we describe the LPAP method, which integrates STGNN-based influence estimation with Actor-Critic-based decision-making to address challenges in dynamic edge environments. Experimentation and evaluation are presented in Section IV. Section V reviews related work. Finally, we conclude this paper in Section VI.

2025 IEEE International Conference on Web Services (ICWS) | 979-8-3315-5563-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICWS67624.2025.00116

2836-3868/25/$31.00 ©2025 IEEE
DOI 10.1109/ICWS67624.2025.00116

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:58 UTC from IEEE Xplore. Restrictions apply.

II. System Model

We model a set of edge servers as $\mathbb{N}$. Each server $n \in \mathbb{N}$ has limited and heterogeneous resources. The available bandwidth between two servers n and n' is denoted as $band_{n,n'}$. The transmission time for a data packet of size ε bytes from n to n' is calculated as

$$T^{tran}(n, n', \varepsilon) = \begin{cases} \frac{\varepsilon}{band_{n,n'}} & \text{if } n \neq n', \\ 0 & \text{if } n = n'. \end{cases} \tag{1}$$

We consider a microservice-based application modeled as a directed acyclic graph (DAG) $\mathbb{G} = \{\mathbb{M}, \mathbb{D}\}$, where $\mathbb{M}$ denotes the set of microservices, and $\mathbb{D}$ denotes the set of dependency edges. Let $v = |\mathbb{M}|$ and $r = |\mathbb{D}|$. The average processing time per request is $T^{proc}(m)$, obtained from historical data.

The application consists of multiple service call chains, represented by $\mathbb{H}$, where each chain $h_i \in \mathbb{H}$ is a sequence of microservices $h_{i,0}, h_{i,1}, \ldots, h_{i,j}$. We introduce a virtual microservice $h_{i,0}$ on each edge node to represent the request entry point, which forwards requests to the actual entry service $h_{i,1}$. The data size transmitted between two consecutive microservices $h_{i,j-1}$ and $h_{i,j}$ is denoted by $w_{h_{i,j-1},h_{i,j}}$. The symbol $\phi_{i,n}$ represents the number of requests for chain h_i arriving at server n.

The instance placement schema is represented by $p_{m,n}$, where $p_{m,n} = \eta$ indicates that η replicas of microservice m are deployed on server n.

To evaluate service performance, we focus on request response time. Given that replicas may be deployed on multiple nodes, communication delays between services must be considered. In this study, $P(m, n)$ denotes the probability that a request is processed by an instance of microservice m deployed on edge server n, which depends on the routing strategy used. Under the round-robin routing strategy, each microservice instance is assigned an equal probability of handling incoming requests. Similarly, $P(h_i)$ denotes the probability that end-users generate requests for the call chain h_i, based on the distribution of requests across edge servers.

To better reflect QoS, we use the 95th percentile response time instead of the average. For each call chain h_i, the 95th percentile response time $T^{p95}(h_i)$ includes both processing and transmission delays along the chain. The application-level response time is defined as the weighted sum over all chains:

$$T^{p95}_{app} = \sum_{h_i \in \mathbb{H}} P(h_i) \cdot T^{p95}(h_i). \tag{2}$$

To ensure effective QoS management, we formulate an optimization objective aimed at minimizing the 95% response time across all call chains. This reflects the goal of quickly recovering from QoS degradation. Formally, the objective is to find the optimal number and placement of replicas such that the overall 95% response time of the application is minimized.

$$\min \quad T^{p95}_{app} \tag{3}$$

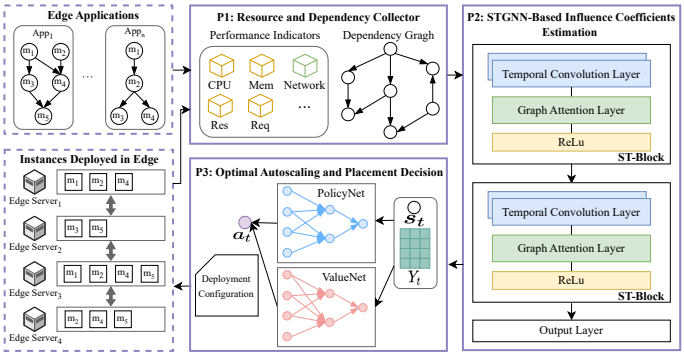


Fig. 1: Workflow of LPAP.

III. Proposed Method

A. LPAP Overview

To address the computational challenges of direct optimization in dynamic edge environments, LPAP introduces an approach that estimates spatial-temporal influence coefficients among microservices and incorporates them into an Actor-Critic-based decision process. As illustrated in Figure 1, LPAP operates in three phases. First, the Resource and Dependency Collector utilizes Prometheus, kube-state-metrics, Istio, and Jaeger to gather key metrics such as tail latency, service invocation traces, resource usage, and network latency, storing the data in a centralized time-series database. Second, a Spatial-Temporal Graph Neural Network is employed to model the microservice system, using temporal convolution layers to capture load dynamics and using graph attention layers to learn inter-service dependencies, ultimately producing an influence coefficient matrix that reflects cross-layer relationships. Third, the Actor-Critic algorithm uses the influence matrix along with system state and network conditions to make adaptive autoscaling and placement decisions, ensuring timely and efficient QoS management. Detailed methods for influence estimation and policy learning are introduced in Section III-B and Section III-C.

B. STGNN-Based Influence Coefficients Estimation

We now present the design and implementation of the STGNN model for estimating influence coefficients, which capture the impact of cross-layer load propagation in microservice applications. These coefficients quantify how load changes along invocation edges affect the performance of target microservices, addressing the challenges of dynamic and diverse resource consumption.

In this paper, we model a microservice-based application as a graph $\mathbb{G} = (\mathbb{M}, \mathbb{D})$, where nodes represent microservices and edges represent invocation relationships. The adjacency matrix $A \in \mathbb{R}^{v \times v}$ defines node connections, and node and edge attributes are described by $\mathrm{X}^t \in \mathbb{R}^{v \times u}$ and $\mathrm{E}^t \in \mathbb{R}^{r \times g}$, respectively. u and g denote the numbers of features associated with each node and edge. At time t, each node m is described by a feature vector $\mathbf{x}^t_m$, which includes metrics such as CPU usage, memory usage, number of replicas, and load. Each edge

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:58 UTC from IEEE Xplore. Restrictions apply.

d_k is characterized by a feature vector $\mathbf{e}_k^t$, which includes call frequency, package size, and latency. The temporal sequences of these features over the past ρ time slices are denoted as $\mathcal{X}^t$ and $\mathcal{E}^t$, respectively. Our goal is to compute the influence coefficient matrix $\mathbf{Y}^t \in \mathbb{R}^{r\times v}$, which captures the impact of edge loads on microservices.

We believe that the scaling decisions of a microservice are influenced by its load as well as the loads of its upstream microservices. Let U_m represent the set of all upstream invocation edges that affect microservice m. Let $y_{i,j}^t$ denote the influence coefficient of the load change on the i-th invocation edge on microservice j, and ultimately, the sum of influence coefficients by μ_m^t for all microservices upstream of m.

$$\mu_m^t = \sum_{d_i \in U_m} y_{i,j}^t \tag{4}$$

In this work, we build a spatial-temporal graph neural network (STGNN) model with several components for estimating influence coefficients.

Temporal Convolution Layer. We use 1-D causal convolutions with Gated Linear Units (GLUs) to capture temporal dynamics in microservice loads. Temporal features from edge and node data are extracted independently. Multi-layer convolutions build hierarchical temporal representations, enabling efficient parallel training. Each layer applies causal convolutions with kernel size K_t, ensuring outputs depend only on past inputs. GLUs act as nonlinear gates modulating the features:

$$T(\nu) = P \odot \sigma(Q) \tag{5}$$

The input tensor $\nu \in \mathbb{R}^{\rho\times\epsilon\times C_i}$ has ρ time steps, ϵ graph elements (nodes or edges), and C_i input channels. Convolution extracts base features $P \in \mathbb{R}^{(\rho-K_t+1)\times\epsilon\times C_o}$ and gating signals $Q \in \mathbb{R}^{(\rho-K_t+1)\times\epsilon\times C_o}$, where K_t is the temporal kernel size and C_o is the number of output channels. The sigmoid gate $\sigma(Q)$ modulates P, producing a temporal feature map.

Graph Attention Layer. Graph Attention Layers (GAT) capture cross-layer load propagation and dependencies. Unlike traditional convolutions, GAT assigns adaptive attention weights based on node features and edge attributes like invocation frequency and latency, focusing on important interactions including distant nodes. The attention coefficient $\alpha_{ij}^{(l)}$ at layer l measures the importance of node j to i:

$$\alpha_{ij}^{(l)} = \frac{\exp\left(\text{ReLU}\left(a^T\left[Wh_i^{(l-1)} \| Wh_j^{(l-1)} \| e_{ij}\right]\right)\right)}{\sum_{k\in N(i)} \exp\left(\text{ReLU}\left(a^T\left[Wh_i^{(l-1)} \| Wh_k^{(l-1)} \| e_{ik}\right]\right)\right)}, \tag{6}$$

where $h_i^{(l-1)}$ and $h_j^{(l-1)}$ are the feature vectors of nodes i and j from layer $l-1$, W is a learnable weight matrix projecting node features into a higher-dimensional space, e_{ij} denotes the edge feature between nodes i and j, a is a learnable attention vector that computes importance scores, and $\|$ denotes concatenation. The ReLU function (commonly implemented as LeakyReLU) introduces non-linearity. The denominator sums over all neighbors k of node i, denoted by $N(i)$, normalizing the coefficients so they sum to 1.

The updated feature vector for node i at layer l is computed as a weighted sum of its neighbors' transformed features:

$$h_i^{(l)} = \sigma\left(\sum_{j\in N(i)} \alpha_{ij}^{(l)} W h_j^{(l-1)}\right), \tag{7}$$

where σ is an activation function ReLU. This aggregation captures both local and long-range dependencies. Incorporating edge features e_{ij} allows the model to consider interaction characteristics like invocation frequency and latency, which are crucial for modeling cross-layer load propagation.

ST-Blocks and Loss Function. We stack two ST-Blocks to capture more complex spatial-temporal dependencies between microservices. The output from these two blocks is then passed through a final output layer to compute the influence coefficients. Specifically, our objective is to compute an influence coefficient matrix $\mathbf{Y}^t \in \mathbb{R}^{r\times v}$, which represents the impact of edge loads on microservices. This matrix is used to guide auto-scaling and placement decisions. The loss function for estimating the influence coefficients, denoted as $\mathcal{L}$, is the L1 loss:

$$\mathcal{L}(Y^t, \hat{Y}^t) = \left|Y^t - \hat{Y}^t\right|^1 \tag{8}$$

where $\hat{Y}^t$ is the training data of the influence coefficient matrix at time step t.

C. Optimal Autoscaling and Placement Decision

The Actor-Critic algorithm combines a policy network (Actor) and a value function (Critic) to optimize decisions in reinforcement learning. In our work, we propose an Actor-Critic-based policy to perform autoscaling and placement for edge-deployed microservices. This policy incorporates an influence coefficient matrix to enhance learning efficiency and decision quality.

a) Markov Decision Process: We model autoscaling and placement in edge environments as a Markov Decision Process (MDP), aiming to minimize response delays and accelerate QoS recovery by optimally placing microservice replicas.

At each decision step t, the system observes a state s_t that captures information such as the available resources of edge nodes including CPU, memory, and network conditions, as well as microservice-level metrics such as resource usage, workload intensity, number of replicas, call frequencies, data volumes, latencies, and current deployment placements. Based on the observed state s_t, the policy π generates an action a_t, which is represented as a placement matrix $p_{m,n}$, indicating the number of instances of microservice m deployed on server n.

To optimize the response time of service chains in microservice-based edge environments, we design a reward function r_t that emphasizes minimizing the response time of chains with higher cumulative influence coefficients. Each microservice m is assigned an influence coefficient μ_m^t, derived from the weighted impact of upstream workloads and request patterns, thereby capturing dynamic propagation effects.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:58 UTC from IEEE Xplore. Restrictions apply.

Let $P(m,n)$ denote the probability that a request is served by microservice m on edge node n, based on a round-robin routing strategy. When m is the entry microservice of chain h_i, this probability reflects the incoming request distribution:

$$P(m,n) = \begin{cases} \frac{p_{m,n}}{\sum_{n\in\mathbb{N}} p_{m,n}} & \text{if } m \neq h_{i,0} \\ \frac{\phi_{i,n}}{\sum_{n\in\mathbb{N}} \phi_{i,n}} & \text{if } m = h_{i,0} \end{cases} \tag{9}$$

Here, $\phi_{i,n}$ denotes the number of requests for service chain h_i received by edge server n.

The request probability of chain h_i, denoted $P(h_i)$, is defined as:

$$P(h_i) = \frac{\sum_{n\in\mathbb{N}} \phi_{i,n}}{\sum_{h_i\in\mathbb{H}} \sum_{n\in\mathbb{N}} \phi_{i,n}}. \tag{10}$$

To better reflect QoS constraints, we evaluate service chain latency using the 95th percentile response time:

$$T^{p95}(h_i) = \sum_{j=1}^{|h_i|} T^{p95}(h_{i,j-1}, h_{i,j}), \tag{11}$$

where $T^{p95}(h_{i,j-1}, h_{i,j})$ represents the 95th percentile processing and transmission time between adjacent services, calculated as:

$$T^{p95}(h_{i,j-1}, h_{i,j}) = T^{p95}_{proc}(h_{i,j}) + \sum_{n\in\mathbb{N}} \sum_{n'\in\mathbb{N}} P(h_{i,j-1}, n) \cdot P(h_{i,j}, n') \cdot T^{p95}_{tran}\left(n, n', w_{h_{i,j-1},h_{i,j}}\right) \tag{12}$$

The importance of each service chain is captured by the normalized influence coefficient $\hat{\lambda}_i$, defined as:

$$\hat{\lambda}_i = \frac{\sum_{j=0}^{|h_i|} \mu^t_{h_{i,j}}}{\sum_{i\in\mathbb{H}} \sum_{j=0}^{|h_i|} \mu^t_{h_{i,j}}}, \tag{13}$$

where μ^t_m denotes the influence from all upstream services in U_m, incorporating the dynamic impact $y^t_{i,j}$ of upstream changes.

The final reward function is the weighted sum of 95th percentile response times across all chains:

$$r_t = \sum_{h_i\in\mathbb{H}} \hat{\lambda}_i \cdot P(h_i) \cdot T^{p95}_{h_i}, \tag{14}$$

where higher weights are assigned to chains with larger influence and traffic volume. This formulation enables the system to prioritize QoS-sensitive chains, minimize worst-case latencies, and accelerate response time recovery. Compared to traditional methods, this approach better accounts for interaction dynamics and cross-layer load propagation, resulting in more effective scaling and placement decisions.

b) Optimizing Microservice Scaling and Placement with Reinforcement Learning: In this section, we present the Actor-Critic with Dynamic Influence Coefficients (AC-DIC) algorithm, a reinforcement learning method designed for microservice autoscaling and placement in edge computing. AC-DIC integrates dynamic influence coefficients to capture load propagation across call chains, enabling more precise and adaptive decisions.

At each step, the system state that includes edge server status, microservice metrics, and network conditions is input to the policy network, which produces scaling and placement actions. The value network evaluates these actions to improve resource utilization and QoS under dynamic conditions. The augmented state $s^{\text{aug}}_t = [s_t, Y_t]$, combining the original state s_t and the influence coefficient matrix Y_t, is used during training to capture complex dependencies, while only s_t is used during transitions to maintain the Markov property.

To speed up learning, we introduce a weighted Temporal Difference (TD) error defined as:

$$\delta_t = \omega_t \cdot \left(r_t + \gamma V_\phi(s^{\text{aug}}_{t+1}) - V_\phi(s^{\text{aug}}_t)\right) \tag{15}$$

where the dynamic weight

$$\omega_t = \sum_{m\in Q_m} \mu^t_m \tag{16}$$

reflects the importance of transitions based on influence coefficients μ^t_m of upstream microservices. Q_m denotes the set of upstream microservices of the current microservice m , including both direct and indirect callers. This weighting focuses training on critical transitions to accelerate convergence.

The value network minimizes the weighted mean squared TD error defined as:

$$\mathcal{L}_V = \frac{1}{|\mathcal{T}|} \sum_t (\delta_t)^2 \tag{17}$$

which improves value estimation by leveraging cross-layer load propagation information within s^{aug}_t.

The policy network updates via the weighted policy gradient as follows.

$$\mathcal{L}_\pi = \frac{1}{|\mathcal{T}|} \sum_t \delta_t \cdot \nabla_\theta \log \pi_\theta(a_t|s^{\text{aug}}_t) \tag{18}$$

and selects actions based on s^{aug}_t during both training and inference. This ensures timely consideration of load propagation effects for informed autoscaling and placement decisions.

After training, the policy π_θ and value V_ϕ networks utilize the augmented state s^{aug}_t to guide decisions, fully accounting for cross-layer load propagation while maintaining computational efficiency in edge environments.

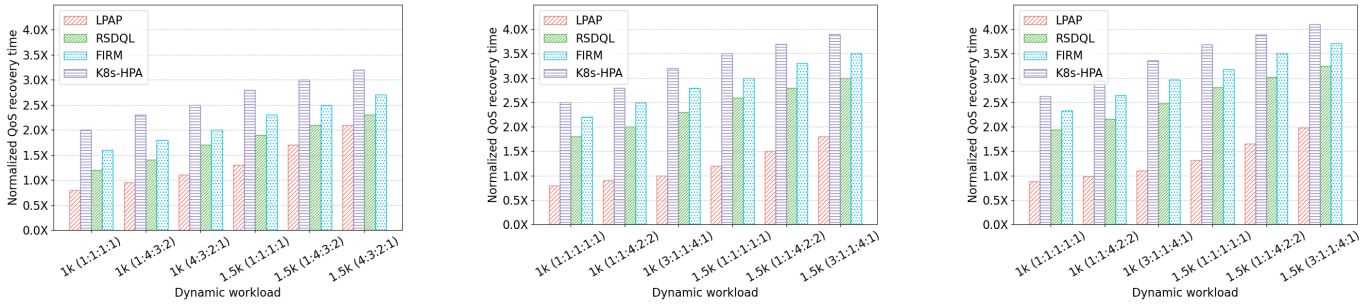
IV. Experiments and Evaluations

In this section, we will evaluate the effectiveness of the proposed LPAP method for edge microservices through extensive experiments. Several sets of experiments have been conducted, and we compare our method with three other representative approaches.

A. Experiment Setup

Experiments were conducted on a private edge cluster comprising eight physical machines, including one master node and seven worker nodes. Each machine was equipped with two Intel(R) Xeon(R) CPU Silve 4314 processors totaling 64 cores, 128 gigabytes of memory, and ran CentOS

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:58 UTC from IEEE Xplore. Restrictions apply.



(a) BookInfo. (b) Online-Boutique. (c) Train-Ticket.

Fig. 2: The normalized QoS recovery time of different benchmarks.

Linux release 7.9. The evaluated methods were implemented on Kubernetes version 1.23.13. To evaluate autoscaling and placement, we selected three open-source microservice applications as benchmarks: (1) BookInfo [10], is an online bookstore application provided by Istio, featuring four integrated microservices named Productpage, Reviews, Details, and Ratings. (2) Online-Boutique [11], is a cloud-native e-commerce demonstration application widely used by Google to showcase Kubernetes, Istio, and gRPC technologies. (3) Train-Ticket [12], simulates a train ticket reservation system with distributed microservices supporting user management, ticket inquiry, and purchase functions.

To simulate production environments with sudden load surges that cause QoS violations, we employed Locust [13], an open-source load testing tool that enables dynamic load scenarios defined in Python. We compared our proposed LPAP method against three baselines: Kubernetes native Horizontal Pod Autoscaler, RSDQL [14], and FIRM [9]. In the STGNN model, we set the Temporal Convolution layer kernel size to three and used two layers in the Graph Attention Network to effectively capture temporal and spatial features. The Adam optimizer with a learning rate of 0.001 was adopted to ensure model stability and convergence. In the AC-DIC algorithm, the discount factor was set to 0.95 and the learning rate to 0.0005 to balance long-term optimization with stable convergence.

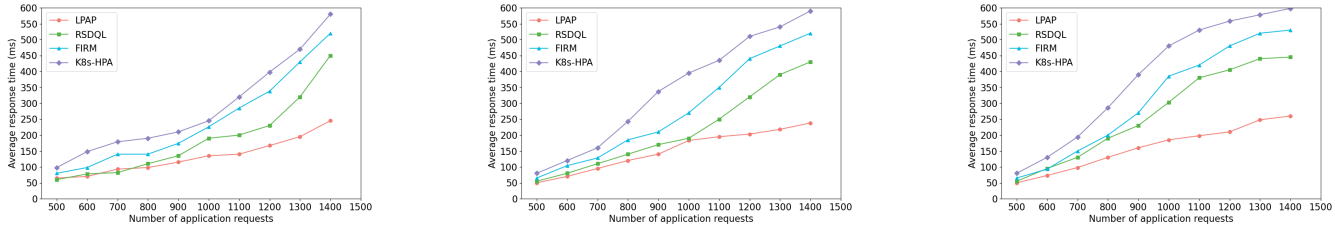
B. Experimental Results

We first stabilized the cluster to handle 300 requests per second (RPS) to establish a steady-state load. Once this stable state was achieved, we introduced sudden surges of traffic to simulate burst load, creating scenarios where QoS violations could occur. The horizontal axis in the figure represents different load configurations, such as *1k (1:1:1:1)* and *1k (1:4:3:2)*. The number *1k* indicates the total number of requests per second, while the ratio *(1:1:1:1)* represents the even distribution of these requests across four service call chains in the BookInfo application. In contrast, the ratio *(1:4:3:2)* means that the requests are distributed unevenly, with a higher proportion of traffic directed towards one or more service chains, reflecting a more realistic scenario in which some chains may receive more traffic than others. We conducted experiments using 4 call chains in BookInfo and 5 invocation chains in Online-Boutique and Train-Ticket.

Figure 2 shows QoS recovery times in six test scenarios in three benchmarks (BookInfo, Online-Boutique, and Train-Ticket), respectively, comparing the performance of LPAP, RSDQL, FIRM, and K8s-HPA. The recovery time is normalized to 3 seconds for easy comparison. In all benchmarks, LPAP consistently demonstrated the lowest QoS recovery time, showing superior adaptability and efficiency in dynamic load scenarios. For example, as shown in Figure 2 in BookInfo, for the uniformly distributed load scenario *1k (1:1:1:1)*, LPAP's normalized recovery time is only 0.8X, significantly better than K8s-HPA (2.0X), RSDQL (1.2X), and FIRM (1.6X). When the load is unevenly distributed, such as *1.5k (4:3:2:1)*, LPAP's recovery time remains at 2.1X, still ahead of other methods. Similarly, as shown in Figure 2 in Online-boutique, LPAP still shows strong robustness and scalability in more complex service dependency scenarios. For example, in the "1k (1:1:4:2:2)" scenario, the normalized recovery time for LPAP is 1.0X, compared to 2.3X for RSDQL, 2.8X for FIRM, and 3.2X for K8s-HPA.

LPAP achieves shorter QoS recovery times by addressing key limitations of existing methods. It dynamically models cross-layer load propagation using a Spatio-Temporal Graph Neural Network (STGNN), ensuring scaling decisions account for changing upstream-to-downstream request ratios and mitigating decision lags. In addition, its integration of autoscaling with placement optimization minimizes network delays, ensuring that replicas are fully utilized. These innovations enable LPAP to recover QoS faster and more reliably than existing methods in dynamic edge environments. This is because K8s-HPA relies on simple periodic load monitoring, often failing to react promptly to dynamic load changes or cross-layer propagation effects. FIRM focuses on resource contention within individual microservices, lacking a holistic view of interactions between microservices and dynamic network conditions at the edge. Moreover, while RSDQL considers dynamic edge environments, it also ignores the cross-layer load propagation. These shortcomings lead to delayed scaling and placement decisions, prolonged QoS violations, and underutilized resources under varying network

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:58 UTC from IEEE Xplore. Restrictions apply.



(a) BookInfo. (b) Online-Boutique. (c) Train-Ticket.

Fig. 3: 95% response time with dynamic loads of different benchmarks.

conditions that LPAP effectively mitigates.

Figure 3 shows the change in the 95% response time of the benchmark applications as the requests grew from 500 to 1400. LAPQE was observed to achieve a shorter 95% response time and the 95% repsonse time increased much slower than the three other methods. Therefore, while achieving fast QoS recovery, nodes can also provide efficient response delay on the premise of ensuring QoS.

V. Related Works

In this section, we review existing works on microservice autoscaling and placement, focusing on how they address resource allocation, replica placement, and the dynamic of edge environments.

A. Microservice Autoscaling

Baarzi and Kesidis [15] proposed a heuristic for vertical and horizontal scaling via resource utilization and latency analysis, using affinity rules to optimize allocation and reduce waste. Qiu et al. [9] introduced an intelligent fine-grained framework for microservices to achieve SLOs, leveraging online telemetry and machine learning to identify critical microservices violating SLOs and reconfigure CPU, memory, etc., for QoS. Meng et al. [16] adopted holistic scaling for dependent microservices, modeling dependencies via Spatio-Temporal GNNs for accurate decisions. Wang et al. [8] used STGNNs for load prediction and CPU utilization optimization to avoid over-provisioning, generating scaling strategies from estimated CPU usage. However, these methods only foucus on autoscaling without considering replica placement, which is not suitable for the actual edge environments.

Cheng et al. [17] proposed QoS optimization and cost management for elastic edge scaling but faced resource shortages during traffic bursts due to cost trade-offs. Cheng et al. [6] applied independent load predictions and moving averages for scaling with burst request strategies. Tong et al. [18] introduced a decentralized collaborative approach via SAC and GMA to address edge-cloud resource heterogeneity. Dimolitsas et al. [19] proposed a multi-application autoscaling framework for Kubernetes edge clusters, optimizing resource allocation and energy consumption via workload prediction and AHP. However, these studies overlook cross-layer load propagation and diverse resource consumption patterns of edge callers.

B. Microservice Placement

Chen et al. [20] proposed a GNN-aided DRL approach for microservice deployment, capturing invocation relationships and edge resource dynamics to improve success rate and utilization. Liu et al. [21] focused on container image layer sharing to reduce download sizes and storage, maximizing edge throughput. Huang et al. [22] jointly optimized microservice dependencies and layer sharing to minimize edge-cloud costs, reducing duplicate image pulls and balancing dependent microservice loads. Afachao et al. [23] introduced a Bi-Generic Advantage Actor-Critic algorithm for edge-cloud deployment optimization, enhancing critic network updates for stability. Farhadi et al. [24] proposed a two-time-scale framework for service placement and scheduling in edge clouds, dynamically adjusting resources for data-intensive applications. He et al. [25] developed a two-stage greedy algorithm to minimize response time in cloud-edge environments, balancing edge load and cloud budgets via dynamic deployment. However, these studies assume fixed replica counts, limiting adaptability to dynamic edge loads.

VI. Conclusion

In this paper, we propose a Load Propagation-aware Autoscaling and Placement (LPAP) approach for Edge QoS Recovery, which jointly optimizes scaling and placement of microservice replicas. By leveraging a Spatio-Temporal Graph Neural Network (STGNN), LPAP estimates influence coefficients that capture the impact of upstream services on target microservices, enabling accurate modeling of cross-layer load propagation. By incorporating these insights with dynamic load patterns, the approach adaptively guides autoscaling and placement decisions. Experimental results show that LPAP significantly shortens QoS violation duration and accelerates recovery compared to existing methods. Future work will further explore dynamic resource allocation for each replica to enhance efficiency and responsiveness at the edge.

VII. Acknowledgment

This work is supported by the National Natural Science Foundation of China (NSFC) under Grant No. 62172050.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:58 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE internet of things journal*, vol. 3, no. 5, pp. 637–646, 2016.

[2] A. R. Sampaio, J. Rubin, I. Beschastnikh, and N. S. Rosa, "Improving microservice-based applications with runtime placement adaptation," *Journal of Internet Services and Applications*, vol. 10, no. 4, pp. 1–30, 2019.

[3] Q. Qu, R. Xu, S. Y. Nikouei, and Y. Chen, "An experimental study on microservices based edge computing platforms," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2020, pp. 836–841.

[4] Y. Li, H. Zhang, W. Tian, and H. Ma, "Joint optimization of auto-scaling and adaptive service placement in edge computing," in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2021, pp. 923–930.

[5] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2021.

[6] K. Cheng, S. Z. Zhang, C. Tu, X. Shi, Z. Yin, S. Lu, Y.-S. Liang, and Q. Gu, "Proscale: Proactive autoscaling for microservice with time-varying workload at the edge," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 4, pp. 1294–1312, 2023.

[7] J. Shi, H. Zhang, Z. Tong, Q. Chen, K. Fu, and M. Guo, "Nodens: Enabling resource efficient and fast qos recovery of dynamic microservice applications in datacenters," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 403–417.

[8] Z. Wang, S. Zhu, J. Li, W. Jiang, K. K. Ramakrishnan, Y. Zheng, M. Yan, X. Zhang, and A. X. Liu, "Deepscaling: microservices autoscaling for stable cpu utilization in large scale cloud systems," in *Proceedings of the 13th Symposium on Cloud Computing*, 2022.

[9] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "Firm: An intelligent fine-grained resource management framework for slo-oriented microservices," in *USENIX Symposium on Operating Systems Design and Implementation*, 2020.

[10] Bookinfo: A microservice benchmark provided by Istio. [Online]. Available: https://istio.io/docs/examples/bookinfo

[11] Online-Boutique: A microservice benchmark used by Google. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo

[12] Train-Ticket: A benchmark microservice system. [Online]. Available: https://github.com/FudanSELab/train-ticket

[13] Locust performance testing tool. [Online]. Available: https://locust.io/

[14] W. Lv, Q. Wang, P. Yang, Y. Ding, B. Yi, Z. Wang, and C. Lin, "Microservice deployment in edge computing based on deep q learning," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2968–2978, 2022.

[15] A. F. Baarzi and G. Kesidis, "Showar: Right-sizing and efficient scheduling of microservices," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 427–441.

[16] C. Meng, S. Song, H. Tong, M. Pan, and Y. Yu, "Deepscaler: Holistic autoscaling for microservices based on spatiotemporal gnn with adaptive graph learning," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 53–65.

[17] K. Cheng, S. Zhang, M. Liu, Y. Gu, L. Wei, H. Cheng, K. Liu, Y. Song, X. Shi, A. Zhu, and L. Tang, "Geoscale: Microservice autoscaling with cost budget in geo-distributed edge clouds," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 4, pp. 646–662, 2024.

[18] G. Tong, C. Meng, S. Song, M. Pan, and Y. Yu, "Gma: Graph multi-agent microservice autoscaling algorithm in edge-cloud environment," in *2023 IEEE International Conference on Web Services (ICWS)*, 2023, pp. 393–404.

[19] I. Dimolitsas, D. Spatharakis, D. Dechouniotis, A. Zafeiropoulos, and S. Papavassiliou, "Multi-application hierarchical autoscaling for kubernetes edge clusters," in *2023 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2023, pp. 291–296.

[20] S. Chen, Q. Yuan, J. Li, H. He, S. Li, X. Jiang, and J. Yang, "Graph neural network aided deep reinforcement learning for microservice deployment in cooperative edge computing," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3742–3757, 2024.

[21] L. Gu, D. Zeng, J. Hu, B. Li, and H. Jin, "Layer aware microservice placement and request scheduling at the edge," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*, 2021, pp. 1–9.

[22] D. Zeng, H. Geng, L. Gu, and Z. Li, "Layered structure aware dependent microservice placement toward cost efficient edge clouds," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*, 2023, pp. 1–9.

[23] K. Afachao, A. M. Abu-Mahfouz, and G. P. Hanke, "Efficient microservice deployment in the edge-cloud networks with policy-gradient reinforcement learning," *IEEE Access*, vol. 12, pp. 133 110–133 124, 2024.

[24] V. Farhadi, F. Mehmeti, T. He, T. F. L. Porta, H. Khamfroush, S. Wang, K. S. Chan, and K. Poularakis, "Service placement and request scheduling for data-intensive applications in edge clouds," *IEEE/ACM Transactions on Networking*, vol. 29, no. 2, pp. 779–792, 2021.

[25] X. He, H. Xu, X. Xu, Y. Chen, and Z. Wang, "An efficient algorithm for microservice placement in cloud-edge collaborative computing environment," *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 1983–1997, 2024.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:54:58 UTC from IEEE Xplore. Restrictions apply.